DEPSTAR
Department of Computer Engineering
Charotar University of Science and Technology
Devang Patel Institute of Advance Technology and Research
Department of Computer Engineering

PRACTICAL 1

➔ Practical Aim

To implement fundamental array operations.

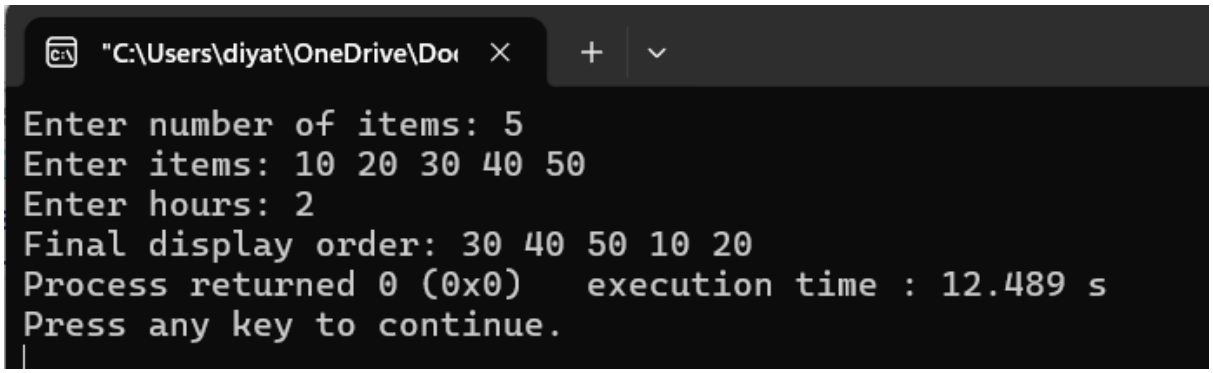
Problem 1

A bakery prepares n items every morning and places them in a display row. At the end of each hour, the leftmost item is moved to the rightmost position to make room for fresh stock at the front. Given the initial row and the number of hours h, print the final display order. Describe the approach you used. How does it behave when h is very large, say 10 million? Is there a way to get the correct result without performing the operation h times?

Source code:

https://github.com/diyatolia1212/FDSA-25DCE115_B-Batch/tree/6a3c760dc37cc4f828716fd7d2fc47aebdfcbbae/PRACTICAL%201

OUTPUT



The program displays the final array after rotating the leftmost item to the rightmost position h times.

Conclusion:

Thus, the fundamental array operations were successfully implemented and the effect of repeated rotations was understood. The concept of using modulo operation was used to handle a very large number of rotations efficiently.

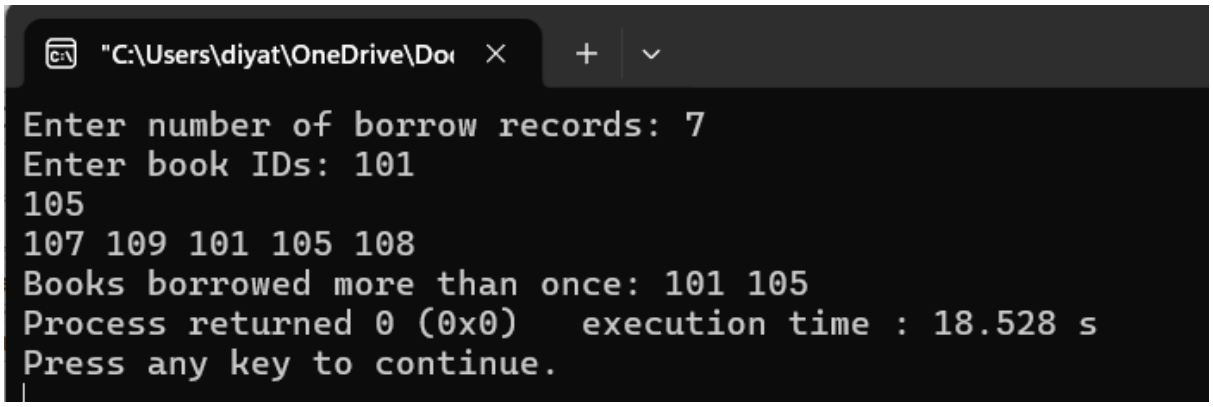
Problem 2

A library issues books to students and records each book's ID every time it is borrowed. At the end of the month, the librarian wants to find all books that were borrowed more than once, as those need priority restocking. Given the borrowing log, print all such book IDs. Describe the approach you used. How many times does your solution scan the data? If the library had 100,000 borrow records, would your approach still be practical? Can you think of away that requires fewer passes?

Source CODE

https://github.com/diyatolia1212/FDSA-25DCE115_B-Batch/tree/6a3c760dc37cc4f828716fd7d2fc47aebdfcbbae/PRACTICAL%201

OUTPUT



The program displays the book IDs that were borrowed more than once.

Conclusion:

Thus, the borrowing records were successfully analyzed to identify book IDs that appeared more than once. The importance of counting occurrences and avoiding duplicate output was understood.

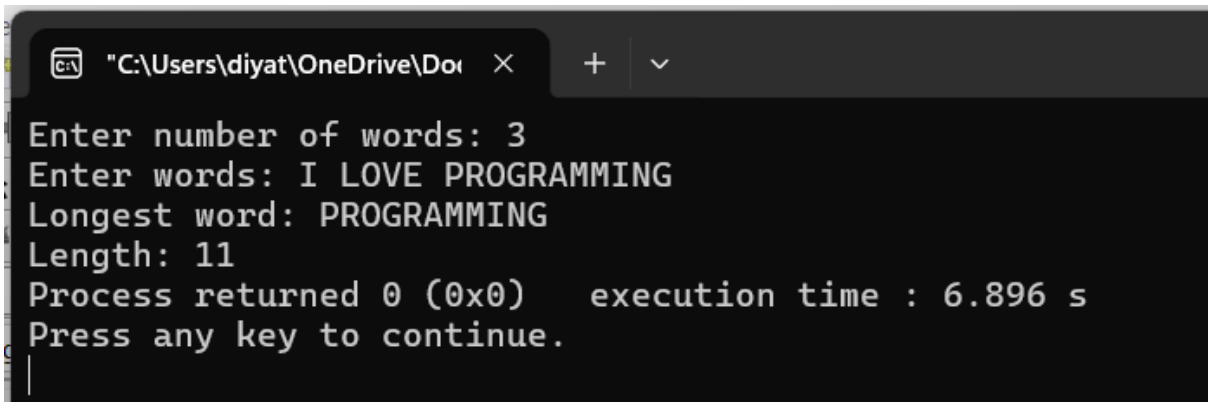
Problem 3

A school newspaper editor wants to highlight the most impressive word from a submitted article on the front page. The word is chosen simply by length — the longest one wins. Given a sentence, print the winning word and how many letters it has. Describe the approach you used. What does your solution do when two words have the same length? Is that behavior intentional, and can you think of a way to handle it explicitly?

Source CODE

https://github.com/diyatolia1212/FDSA-25DCE115_B-Batch/tree/6a3c760dc37cc4f828716fd7d2fc47aebdfcbbae/PRACTICAL%201

OUTPUT



The program displays the longest word in the given sentence along with its length.

Conclusion:

Thus, the longest word in a given sentence was successfully identified along with its length. The handling of words with equal length and punctuation was also understood.

Challenges Faced :

PROBLEM 1.A

1. Wrong Rotation Logic : cout << items[(i + h) % n];
   Correction : int rotations = h % n;
   cout << items[(i + rotations) % n];
2. Wrong Loop Condition : for(int i=0;i<=n;i++) this loop accesses items [n]
   Correction: for(int i=0;i<n;i++)

PROBLEM 1.B

1. Wrong loop condition: for(int j=0;j<=n;j++)
   Correction: for(int j=0;j<n;j++) …As I was writing j<=n when the correct form is j<n so due to that array index was going beyond the limit.

2. Some common errors like missing braces , undeclared variable and printing duplicates multiple times.

PROBLEM 1.C

1. String Not Declared
   Correction: #include<string>
   string word;
2. Wrong Comparison : if(word > longest)
   Correction: if(word.length() > longest.length())

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. [Problem 1] The display order repeats itself after a certain number of hours regardless of how large h gets. Without trying any specific example, explain why this repetition is inevitable and what determines when it first repeats.

Answer: The display order repeats after n hours because each hour the array is rotated left by one position. After n rotations, every element returns to its original position. The repetition period is determined by the number of items n. Therefore, for a very large h, we can use h % n to find the effective number of rotations.

2. [Problem 2] If the same book ID appears three times in the log, should it appear once or three times in your output? Does your solution handle this correctly, and how would you verify it?

Answer: If the same book ID appears three times, it should appear only once in the output because the requirement is to find book IDs that were borrowed more than once. The solution should count the occurrences of each ID and print an ID only when its count is greater than 1. We can verify this by checking that no ID is printed more than once.

3. [Problem 3] The editor later decides that punctuation attached to a word (e.g. “hello,” or “great!”) should not count toward its length. How would this change your solution, and at which step would you handle it?
Answer: If punctuation should not count toward the word length, punctuation marks should be removed before calculating the length of each word. This can be handled while processing each word, before comparing its length with the current longest word.